

PRAXISLEITFADEN

Umzug auf einen neuen Windows-Rechner

Pool Control Display · ESP32-S3 · PlatformIO · MQTT/LoxBerry

Ziel

Den aktuellen Projektstand vollständig und reproduzierbar auf den neuen Rechner übertragen, ohne lokale Zugangsdaten zu verlieren und ohne unnötige Build- oder Cachedateien mitzunehmen.

Die drei wichtigsten Regeln

- [] Quellcode über GitHub übertragen: alten Stand committen und pushen, anschließend frisch klonen.
- [] Die private `include\PoolConfig.h` separat sichern oder neu anlegen. Sie ist absichtlich nicht in Git.
- [] Die Werksdemo des Panels vor dem ersten eigenen Upload prüfen und dokumentieren.

Über GitHub	Quellcode, Dokumentation, Tests, Boarddefinition und CI-Konfiguration
Separat	WLAN-/MQTT-Daten, Fotos, serielle Logs und lokale Notizen
Neu erzeugen	PlatformIO-Pakete, <code>.pio</code> , VS-Code-Caches und Build-Artefakte

Zusätzlich enthalten: eine separate Textdatei mit allen Befehlen, Erklärungen, Erwartungen und Sicherheitshinweisen.

1. Alten Rechner sauber abschließen

Arbeitsstand prüfen

Vor dem Rechnerwechsel muss klar sein, welche Änderungen nur lokal vorhanden sind.

```
git status
```

```
git diff
```

Aktueller Projektstand

Der Branch ist main. Die Übergabedatei CODEX_HANDOFF.md enthält aktuell noch nicht eingetragene Änderungen.

Änderung gezielt committen und übertragen

```
git add CODEX_HANDOFF.md
```

```
git commit -m "Docs: Windows migration handoff"
```

```
git push origin main
```

Danach erneut `git status` ausführen. Der Arbeitsbaum sollte sauber sein und main sollte dem Stand von origin/main entsprechen.

Private und lokale Inhalte sichern

- [] WLAN-SSID und WLAN-Passwort
- [] MQTT-Host, Port, Benutzername und Passwort
- [] Optionales OTA-Passwort, falls OTA später aktiviert wird
- [] Fotos der Werksdemo, PCB-Revision, Touch-Prüfung und serielle Logs

Hinweis zu PoolConfig.h

Im derzeitigen Arbeitsverzeichnis existiert `include\PoolConfig.h` nicht. Die echten Werte müssen daher aus einer anderen sicheren Quelle übernommen oder neu eingetragen werden. Zugangsdaten niemals committen.

Nicht kopieren

Die Ordner `.pio`, Python-Caches, alte Build-Artefakte und lokale VS-Code-Caches sind rechnerabhängig oder generiert. Sie werden auf dem neuen Rechner neu erzeugt.

2. Neuen Windows-Rechner vorbereiten

Benötigte Software

- [] Git für Windows einschließlich Git Credential Manager
- [] Visual Studio Code
- [] VS-Code-Erweiterung PlatformIO IDE
- [] Python 3 mit aktivierter Option 'Add Python to PATH'
- [] Optional: MSYS2/MinGW oder ein anderer C++-Compiler für lokale native Tests

Der normale ESP32-Build benötigt keinen separat installierten ESP32-Compiler: PlatformIO lädt die festgelegten Toolchains selbst. Ein Windows-C++-Compiler wird nur für `pio test -e native` benötigt.

Installation kurz prüfen

```
git --version
```

```
python --version
```

Repository frisch klonen

```
git clone https://github.com/noncon66/pool-control-display.git
```

```
Set-Location .\pool-control-display
```

```
git status
```

```
code .
```

GitHub-Zugriff

Für das Klonen eines öffentlichen Repositorys ist keine Anmeldung nötig. Für spätere Pushes muss die GitHub-Anmeldung über den Credential Manager eingerichtet sein.

Private Gerätekonfiguration anlegen

```
Copy-Item .\include\PoolConfig.example.h .\include\PoolConfig.h
```

Anschließend `PoolConfig.h` in VS Code öffnen und die echten WLAN- und MQTT-Werte eintragen. OTA zunächst deaktiviert lassen.

```
Test-Path .\include\PoolConfig.h
```

Erwartetes Ergebnis: True. Der Prüfcommand zeigt keine Zugangsdaten an.

3. Installation und Projekt verifizieren

PlatformIO prüfen

```
pio --version
```

Falls der Befehl nach Installation der VS-Code-Erweiterung noch nicht gefunden wird, VS Code und das Terminal neu starten. Alternativ den ersten Build über die PlatformIO-Oberfläche in VS Code ausführen.

Firmware bauen - ohne Upload

```
pio run -e esp32-s3-panel
```

Der erste Build lädt die festgelegten ESP32-Pakete und Bibliotheken. Er kann länger dauern, verändert das Panel aber nicht. Erwartet wird SUCCESS.

Hardwareunabhängige Tests

```
pio test -e native
```

Die Tests prüfen unter anderem Statusmodell, Frische, Bedienfreigaben und Befehlsbestätigung. Fehlt unter Windows ein C++-Compiler, können die Tests weiterhin über GitHub Actions ausgeführt werden.

Python-Werkzeuge gefahrlos prüfen

```
python .\tools\loxone_mqtt_simulator.py --help
```

```
python .\tools\display_bringup.py --help
```

Diese Hilfeaufrufe verbinden sich nicht mit dem Broker und flashen keine Hardware.

Optional: MQTT-Selbsttest

```
python .\tools\loxone_mqtt_simulator.py --broker 127.0.0.1 --self-test
```

Nur verwenden, wenn lokal bereits ein MQTT-Testbroker auf Port 1883 läuft. Für den Umzug ist dieser Schritt optional; die GitHub Actions führen einen isolierten Brokertest aus.

Netzwerkcheck

Der neue Rechner muss LoxBerry beziehungsweise den MQTT-Broker im lokalen Netz erreichen können. Typisch ist TCP-Port 1883. Netzwerksegment, Firewall und Broker-Zugangsdaten müssen stimmen.

4. Panel sicher anschließen und testen

Vor jedem eigenen Upload

- [] USB-C-Datenkabel verwenden und das Panel am Anschluss 'USB TO UART' verbinden.
- [] Unveränderte Werksdemo auf Funktion prüfen und fotografieren.
- [] Touch-Funktion testen und Ergebnis dokumentieren.
- [] PCB-Aufdruck Rev2.2 und den unter Windows erkannten COM-Port notieren.

COM-Port finden

```
pio device list
```

Alternativ im Windows-Geräte-Manager unter 'Anschlüsse (COM & LPT)' nachsehen. Nur wenn kein Port erscheint, den von Waveshare empfohlenen USB-UART-Treiber installieren.

Isolierten Displaytest bauen

```
python .\tools\display_bringup.py build
```

Dieser Build verwendet ein separates PlatformIO-Core-Verzeichnis und bleibt von der normalen Firmware getrennt. Er flasht noch nichts.

Erst nach dokumentierter Werksdemo hochladen

```
python .\tools\display_bringup.py upload --port COM5
```

```
python .\tools\display_bringup.py monitor --port COM5
```

COM5 ist ein Beispiel

Beide Befehle müssen den tatsächlich erkannten COM-Port verwenden. Der Upload überschreibt die Werksfirmware. Der isolierte Test initialisiert nur das Display - nicht Touch, LVGL, WLAN, MQTT oder Poolsteuerung.

Erwartetes Ergebnis

Der Bildschirm zeigt eine weiße Fläche mit rotem und schwarzem Testtext. Zusätzlich Bildschirmfoto und serielles Log sichern. Den Monitor mit Strg+C beenden.

5. Abschlusskontrolle

Umzug abgeschlossen, wenn ...

- [] Das Repository frisch von GitHub geklont wurde und `git status` sauber ist.
- [] Die `private PoolConfig.h` lokal existiert und echte Werte enthält.
- [] Der normale Firmwarebuild erfolgreich endet.
- [] Die nativen Tests lokal oder in GitHub Actions erfolgreich sind.
- [] Beide Python-Hilfaufrufe funktionieren.
- [] Der MQTT-Broker vom neuen Netzwerk aus erreichbar ist.
- [] Werksdemo, Touch, PCB-Revision und COM-Port vor einem Upload dokumentiert wurden.

Schnelle Diagnosebefehle

Get-Command git, python, pio

Zeigt, ob und von welchem Speicherort PowerShell die Programme startet.

```
git remote -v
```

Zeigt die GitHub-Adresse für Abruf und Push.

```
git log -1 --oneline
```

Zeigt den neuesten lokalen Commit zum Vergleich mit dem alten Rechner und GitHub.

Projektdateien

Die technische Referenz bleibt im Repository: `README.md`, `CODEX_HANDOFF.md`, `docs/hardware.md`, `docs/ci.md` und `docs/simulator.md`.

Ende der Checkliste